At the moment, users must log in via a browser with a username and password. Since Keycloak now manages user accounts, we could go ahead and update Edge Service to check the user credentials with Keycloak itself, rather than using its internal storage. But what happens if we introduce different clients to the Polar Bookshop system, such as mobile applications and IoT devices? How should the users authenticate then? What if the bookshop employees are already registered in the company's Active Directory (AD) and want to log in via SAML? Can we provide a single sign-on (SSO) experience across different applications? Will the users be able to log in via their GitHub or Twitter accounts (*social login*)?

We could think of supporting all those authentication strategies in Edge Service as we get new requirements. However, that is not a scalable approach. A better solution is delegating a dedicated *identity provider* to authenticate users following any supported strategy. Edge Service would then use that service to verify the identity of a user without being concerned about performing the actual authentication step. The dedicated service could let users authenticate in various ways, such as using the credentials registered in the system, through social login, or via SAML to rely on the identity defined in the company's AD.

Using a dedicated service to authenticate users leads to two aspects we need to address for the system to work. First, we need to establish a protocol for Edge Service to delegate user authentication to the identity provider and for the latter to provide information about the authentication result. Second, we need to define a data format that the identity provider can use to securely inform Edge Service about the identity of users after they have been successfully authenticated. This section will address both issues using OpenID Connect and JSON Web Token.

11.3.1 Authenticating users with OpenID Connect